SCS 1103 · OPERATING SYSTEMS · PROCESS MANAGEMENT

Operating Systems and CPU Scheduling

What an operating system does, processes and threads, schedulers and scheduling criteria, then FCFS, SJF, SRTF, Priority and Round Robin worked to the last millisecond with Gantt charts. Also deadlocks, paging, spooling, multiprogramming and locality: every question on the SCS 1103 November 2024 paper, answered.

WHITEPAPER · SCHEDULING AND OS

COURSE
SCS 1103 Operating Systems (also SCS 1101 Module 4)

BUILT ON
NUST SCS 1103 Final Examination, November 2024

CALCULATIONS
Every Gantt chart and average computed by a simulator, not by hand

PREPARED FOR
Group 13 · NUST Civil & Water Engineering

INSIDE THIS WHITEPAPER

1. What an operating system is and does
2. Processes, PCB, states and threads
3. Multiprogramming, jobs and pseudo-parallelism
4. Schedulers, dispatcher and criteria
5. The formulas and how to work a question
6. FCFS
7. SJF and SRTF
8. Priority scheduling
9. Round Robin
10. Multilevel queues and comparison
11. Nov 2024 Q3 fully solved
12. Deadlocks
13. Memory: paging
14. Spooling, P2P vs client-server, locality
15. Exam questions and spotter

Theory follows Silberschatz, Galvin and Gagne (Operating System Concepts) and Tanenbaum (Modern Operating Systems), the two standard texts for this course.

Group 13
Study Board

1 What an operating system is and does

An **operating system (OS)** is the program that manages the computer's hardware and provides services and an environment in which application programs run. It sits between the user and applications on one side and the hardware on the other. Examples: Windows, Linux, macOS, Android, iOS.

Three main purposes (Nov 2024 Q1a, 6 marks)

1. **Convenience**: makes the computer easy to use by hiding hardware details behind a simple interface (the OS as an **extended machine**).
2. **Efficiency / resource manager**: allocates the CPU, memory, disk and devices fairly and efficiently between competing programs and users.
3. **Ability to evolve / control program**: controls the execution of programs to prevent errors and misuse, and allows new functions to be added without disrupting service.

Main functions

- **Process management**: create, schedule, suspend and terminate processes
- **Memory management**: allocate and protect memory; virtual memory
- **File management**: files, directories, permissions
- **Device (I/O) management**: drivers, buffering, spooling
- **Security and protection**: authentication, access control
- **User interface**: command line or GUI
- **Networking and accounting**

Kernel: the core part of the OS that is always in memory and runs in **kernel (privileged) mode**; user programs run in **user mode** and request services through **system calls** (for example `read()`, `fork()`).

2 Processes, the PCB, process states and threads

A **program** is a passive file of instructions on disk. A **process** is a program **in execution**: the code plus its current activity (program counter, registers), stack, data and heap. One program can produce several processes (two people running the same browser).

Process Control Block (PCB)

The OS keeps one PCB per process, holding: process ID and state; program counter; CPU registers; scheduling information (priority, queue pointers); memory-management information (page tables); accounting information (CPU time used); I/O status (open files, devices). On a **context switch** the OS saves the running process's state in its PCB and loads the next one's. Switching takes time and does no useful work, so it is pure overhead.

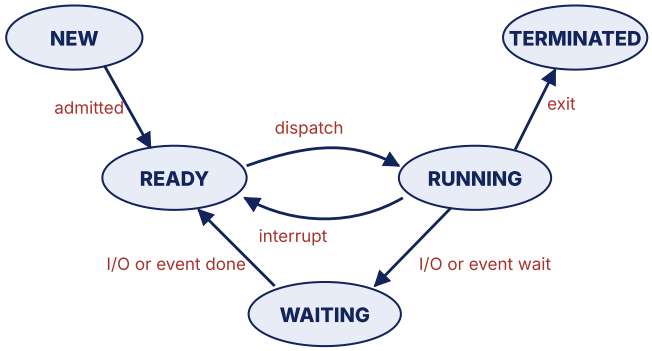


Figure 1. The five-state process model.

State	Meaning
New	The process is being created.
Ready	Waiting in the ready queue to be given the CPU.
Running	Its instructions are being executed (one process per CPU core at a time).
Waiting (blocked)	Waiting for an event such as I/O completion; cannot use the CPU even if free.
Terminated	Finished; its resources are released.

Threads. A **thread** is the basic unit of CPU use inside a process: it has its own program counter, registers and stack, but **shares** code, data and open files with the other threads of the same process. **Benefits of multithreaded programming** (Nov 2024 Q2b, 5 marks): (1) **responsiveness**: one thread keeps the interface working while another does a long task; (2) **resource sharing**: threads share memory without special arrangements; (3) **economy**: creating and switching threads is much cheaper than processes; (4) **scalability**: threads run in parallel on multicore CPUs; (5) simpler program structure for tasks that are naturally concurrent (a server handling many clients).

3 Multiprogramming, jobs and pseudo-parallelism

Multiprogramming keeps several jobs in memory at once. When the running job has to wait (for example for I/O), the OS switches the CPU to another job instead of leaving it idle. **Advantages** (Q4b): (1) higher **CPU utilisation**; (2) higher **throughput** (more jobs finished per hour); (3) shorter average **response and turnaround times**; (4) better use of memory and I/O devices running in parallel with the CPU; (5) several users or programs appear to run at once; (6) supports time-sharing and interactive systems. Disadvantages: needs memory management, protection and scheduling, so the OS is more complex.

Job vs process (Q4a, 5 marks). A **job** is a unit of work submitted to the system, typically in batch systems: a program with its data and control instructions, waiting in a job pool on disk to be admitted. A **process** is the active entity created when a job (or any program) is loaded into memory and begins executing, with its own PCB, state and resources. A job may consist of several processes; "job" is the user's or batch view, "process" is the OS's execution view. Older texts use the terms almost interchangeably (hence "job scheduler").

Pseudo-parallelism (Q5a i, 2 marks): on a single CPU only one process runs at any instant, but the OS switches between processes so quickly (every few milliseconds) that they **appear** to run at the same time. True parallelism needs several CPUs or cores. **The process model** (Q5a ii, 8 marks; Tanenbaum): all runnable software, including the OS's own services, is organised as a set of **sequential processes**, each with its own **virtual CPU** (its own program counter, registers and variables). Conceptually each process runs independently; in reality the one real CPU switches between them (multiprogramming). This makes parallelism easier to reason about because the programmer thinks about each process as a simple sequential program and leaves the switching to the OS. The model must avoid assumptions about timing, because a process's speed depends on when it is switched out; processes cooperate through well-defined mechanisms (pipes, messages, semaphores).

4 Schedulers, the dispatcher and scheduling criteria

Scheduler	Job	How often
Long-term (job scheduler)	Chooses which jobs from the job pool are admitted into memory; controls the **degree of multiprogramming** and the mix of CPU-bound and I/O-bound jobs.	Seconds to minutes
Short-term (CPU scheduler)	Chooses which ready process gets the CPU next.	Every few ms; must be fast
Medium-term	**Swaps** processes out of memory to disk and back to reduce the load.	Occasionally

What is process scheduling? (Q4c i, 2 marks) The activity of the OS that decides **which process runs on the CPU, when, and for how long**, by selecting processes from the ready queue according to a scheduling algorithm. The **dispatcher** then gives the CPU to the chosen process: it performs the context switch, switches to user mode and jumps to the right place in the program. The time this takes is the **dispatch latency**.

Preemptive vs non-preemptive. In **non-preemptive** scheduling a process keeps the CPU until it finishes or blocks (FCFS, SJF, non-preemptive priority). In **preemptive** scheduling the OS can take the CPU away (when

a time slice ends or a more important process arrives: RR, SRTF, preemptive priority).

Scheduling criteria (Q4c ii)

- **CPU utilisation**: keep the CPU busy (maximise)
- **Throughput**: processes completed per unit time (maximise)
- **Turnaround time**: submission to completion (minimise)
- **Waiting time**: total time spent in the ready queue (minimise)
- **Response time**: submission to first response (minimise; key for interactive users)

Characteristics of a good scheduler

- **Fairness**: every process gets a fair share; no starvation
- **Efficiency**: high utilisation and throughput, low overhead
- **Predictability**: similar jobs take similar times
- **Policy enforcement**: respects priorities
- **Balance**: keeps CPU and I/O devices busy (mix of CPU-bound and I/O-bound)
- Low **dispatch latency** and few unnecessary context switches

Factors that affect performance: the algorithm, the length of the time quantum, context-switch overhead, the mix of CPU-bound and I/O-bound processes, arrival pattern and burst lengths, and the number of processors.

5 The formulas and how to work a question

> **LEARN THESE THREE LINES**
>
> **Turnaround time (TAT) = Completion time − Arrival time**
> **Waiting time (WT) = Turnaround time − Burst time**
> **Response time = First run time − Arrival time**
> When every process arrives at time 0, TAT = completion time, and for non-preemptive algorithms WT = start time. Average = sum ÷ number of processes.

1. Copy the table; note arrival times (assume 0 if not given) and the tie rule (equal values go in table order, FCFS).
2. Decide the run order using the algorithm's rule.
3. Draw the **Gantt chart**: a bar split into blocks labelled with the process, with times written **under every boundary** starting at 0.
4. Fill a table with completion, TAT and WT for each process.
5. Calculate the averages, showing the sum. Give units (ms).
6. Comment: which algorithm gave the lowest average and why.

6 First Come, First Served (FCFS)

Rule: processes are served in the order they arrive, using a FIFO queue. **Non-preemptive. Advantages**: simplest to understand and implement; fair in arrival order; no starvation. **Disadvantages**: average waiting time is often long; the **convoy effect** (short processes stuck behind one long CPU-bound process, as P2–P4 wait behind P1 below); poor for interactive systems.

Worked example (the class notebook question): P1 = 8, P2 = 3, P3 = 5, P4 = 2 ms, all arriving at 0.



Process	Burst	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	8	8	8	0
P2	3	11	11	8
P3	5	16	16	11
P4	2	18	18	16

Average waiting time = (0 + 8 + 11 + 16) ÷ 4 = **8.75 ms** · Average turnaround = (8 + 11 + 16 + 18) ÷ 4 = **13.25 ms**

7 Shortest Job First (SJF) and Shortest Remaining Time First (SRTF)

SJF rule: give the CPU to the process with the **smallest next CPU burst**; ties go FCFS. **Non-preemptive.** SJF is **provably optimal**: it gives the minimum average waiting time for a given set of processes. **Disadvantages**: the length of the next burst is not known in advance (it is estimated, usually by exponential averaging of past bursts: $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$); long processes can **starve** if short ones keep arriving.

Same notebook processes with SJF (compare 8.75 ms for FCFS)



Process	Burst	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	8	18	18	10
P2	3	5	5	2
P3	5	10	10	5
P4	2	2	2	0

Average waiting time = (10 + 2 + 5 + 0) ÷ 4 = **4.25 ms** · Average turnaround = (18 + 5 + 10 + 2) ÷ 4 = **8.75 ms**

SRTF is the **preemptive** version of SJF: when a new process arrives with a burst shorter than the **remaining** time of the running process, the running process is preempted. It gives even lower average waiting time when processes arrive at different times, but has more context switches.

Arrival-time example: P1 (arr 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5)

Non-preemptive SJF: P1 starts alone at 0 and must finish before anyone else runs.



Process	Arrival	Burst	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	0	8	8	8	0
P2	1	4	12	11	7
P3	2	9	26	24	15
P4	3	5	17	14	9

Average waiting time = (0 + 7 + 15 + 9) ÷ 4 = **7.75 ms** · Average turnaround = (8 + 11 + 24 + 14) ÷ 4 = **14.25 ms**

SRTF: at t = 1, P2 (4) is shorter than P1's remaining 7, so P1 is preempted.



Process	Arrival	Burst	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Average waiting time = (9 + 0 + 15 + 2) ÷ 4 = **6.5 ms** · Average turnaround = (17 + 4 + 24 + 7) ÷ 4 = **13 ms**

For the same data FCFS gives 8.75 ms. So FCFS 8.75 → SJF 7.75 → SRTF 6.5 ms.

8 Priority scheduling

Rule: each process has a priority number; the CPU goes to the **highest priority**. In this course and in Silberschatz, a **low number means high priority** (1 is highest); say so in your answer. It can be **non-preemptive** or **preemptive** (a newly arrived higher-priority process takes the CPU). SJF is a special case where priority = predicted burst. **Problem**: **starvation** (indefinite blocking) of low-priority processes. **Solution**: **ageing**, gradually raising the priority of processes that have waited a long time.

9 Round Robin (RR)

Rule: FCFS with **preemption** after a fixed **time quantum** (time slice), typically 10–100 ms. The ready queue is circular: a process that has not finished after its quantum goes to the **back** of the queue. Designed for **time-sharing**. **Advantages**: fair; no starvation; good response time; every process gets 1/n of the CPU. **Disadvantages**: higher average turnaround than SJF; performance depends on the quantum: **too large** → behaves like FCFS; **too small** → too many context switches (overhead). Rule of thumb: about 80% of CPU bursts should be shorter than the quantum.

Notebook processes with Round Robin, quantum = 3 ms



Process	Burst	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	8	18	18	10
P2	3	6	6	3
P3	5	16	16	11
P4	2	11	11	9

Average waiting time = (10 + 3 + 11 + 9) ÷ 4 = **8.25 ms** · Average turnaround = (18 + 6 + 16 + 11) ÷ 4 = **12.75 ms**

Queue rule used: when a quantum ends at the same moment a new process arrives, the new arrival joins the queue **before** the preempted process. State the rule you use in the exam.

10 Multilevel queues and comparison

Multilevel queue: the ready queue is split into several queues (for example system, interactive, batch), each with its own algorithm (RR for interactive, FCFS for batch) and fixed priority between queues. **Multilevel feedback queue (MLFQ)**: processes can **move** between queues: a process that uses its whole quantum

drops to a lower queue; one that waits too long moves up (ageing). It is the most general and most complex scheme and is close to what real operating systems use.

Algorithm	Preemptive?	Strength	Weakness	Starvation?
FCFS	No	Simple, fair by arrival	Convoy effect, long average wait	No
SJF	No	Minimum average waiting time	Burst length must be predicted	Yes, long jobs
SRTF	Yes	Even lower wait with arrivals	Overhead; prediction needed	Yes
Priority	Either	Important work first	Indefinite blocking	Yes (fix: ageing)
Round Robin	Yes	Fair, good response time	Quantum choice; higher turnaround	No
MLFQ	Yes	Adapts to process behaviour	Complex to tune	Prevented by ageing

11 SCS 1103 November 2024, Question 3(b): fully solved

THE QUESTION [10 MARKS]

For the processes below, calculate the average waiting time for First Come First Serve (FCFS), Shortest Job First (SJF) and Priority Scheduling. Burst times in ms; all arrive at time 0 in the order P1–P5; a lower number means higher priority.

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

(i) FCFS



Process	Burst	Priority	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	10	3	10	10	0
P2	1	1	11	11	10
P3	2	4	13	13	11
P4	1	5	14	14	13
P5	5	2	19	19	14

Average waiting time = (0 + 10 + 11 + 13 + 14) ÷ 5 = **9.6 ms** · Average turnaround = (10 + 11 + 13 + 14 + 19) ÷ 5 = **13.4 ms**

(ii) SJF (P2 and P4 tie at 1 ms: P2 goes first by table order)



Process	Burst	Priority	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	10	3	19	19	9
P2	1	1	1	1	0
P3	2	4	4	4	2
P4	1	5	2	2	1
P5	5	2	9	9	4

Average waiting time = (9 + 0 + 2 + 1 + 4) ÷ 5 = **3.2 ms** · Average turnaround = (19 + 1 + 4 + 2 + 9) ÷ 5 = **7 ms**

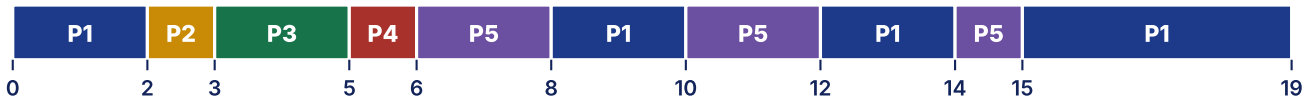
(iii) Priority (non-preemptive; 1 = highest)



Process	Burst	Priority	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	10	3	16	16	6
P2	1	1	1	1	0
P3	2	4	18	18	16
P4	1	5	19	19	18
P5	5	2	6	6	1

Average waiting time = (6 + 0 + 16 + 18 + 1) ÷ 5 = **8.2 ms** · Average turnaround = (16 + 1 + 18 + 19 + 6) ÷ 5 = **12 ms**

Bonus: Round Robin, quantum = 2 ms



Process	Burst	Priority	Completion	Turnaround (C − A)	Waiting (TAT − B)
P1	10	3	19	19	9
P2	1	1	3	3	2
P3	2	4	5	5	3
P4	1	5	6	6	5
P5	5	2	15	15	10

Average waiting time = (9 + 2 + 3 + 5 + 10) ÷ 5 = **5.8 ms** · Average turnaround = (19 + 3 + 5 + 6 + 15) ÷ 5 = **9.6 ms**

CONCLUSION TO WRITE

Average waiting times: FCFS 9.6 ms, SJF 3.2 ms, Priority 8.2 ms (RR q = 2: 5.8 ms). SJF gives the lowest average waiting time because short processes run first, so fewer processes wait behind long ones; FCFS is worst here because of the convoy effect behind P1 (10 ms).

12 Deadlocks

A **deadlock** is a situation in which a set of processes are blocked forever, each holding a resource and waiting for a resource held by another process in the set. Example: P1 holds the printer and waits for the

scanner; P2 holds the scanner and waits for the printer.

Four necessary conditions (Coffman, 1971); all four must hold at once: (1) **mutual exclusion**: a resource can be used by only one process at a time; (2) **hold and wait**: a process holds at least one resource while waiting for others; (3) **no preemption**: resources cannot be taken away, only released voluntarily; (4) **circular wait**: a closed chain P1 → P2 → ... → Pn → P1, each waiting for the next.

Method (Q1b, 9 marks)	How it works
Prevention	Make one of the four conditions impossible: share resources where possible; require a process to request all resources at once (no hold and wait); allow preemption of resources; impose a **numbered ordering** on resources and request them in increasing order (no circular wait).
Avoidance	The OS uses advance information about maximum needs and grants a request only if the system stays in a **safe state**. The **Banker's algorithm** (Dijkstra) is the standard method; a resource-allocation graph is used for single instances.
Detection and recovery	Allow deadlocks, detect them periodically with a wait-for graph or detection algorithm, then recover by **terminating** processes (all, or one at a time) or **preempting** resources and rolling processes back.
Ignore it (ostrich algorithm)	Assume deadlocks are rare and let the user restart; used by most general-purpose systems such as Windows and Linux because prevention is expensive.

13 Memory management: paging

Paging (Q2c, 16 marks) is a memory-management scheme that allows a process's physical memory to be **non-contiguous**. Physical memory is divided into fixed-size blocks called **frames**; logical memory is divided into blocks of the same size called **pages** (usually 4 KB). Any page can be placed in any free frame, so **external fragmentation** is eliminated.

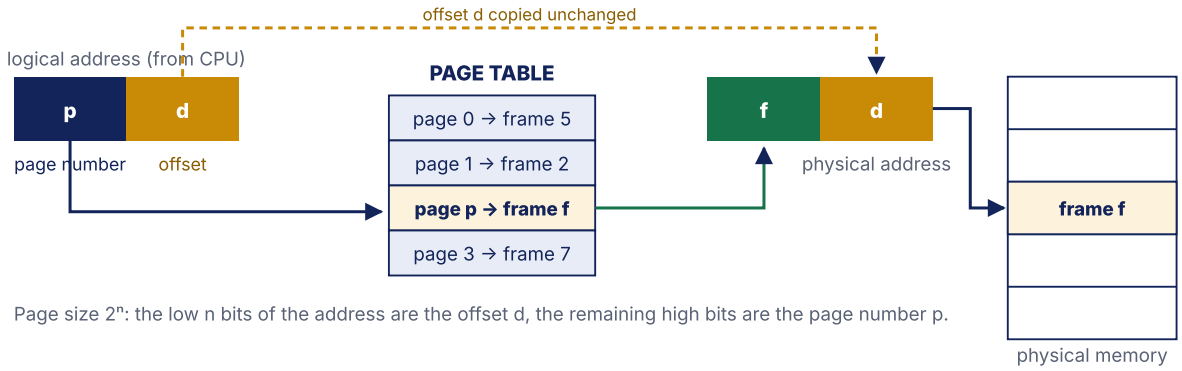


Figure 2. Paging hardware: the page number indexes the page table to find the frame; the offset is added unchanged.

- **Address translation**: every logical address = (page number p, offset d). The page table maps p to frame f; physical address = f × page size + d. **Example**: page size 1 KB (1024 B); logical address 3000 → p = 3000 div 1024 = 2, d = 3000 mod 1024 = 952. If page 2 is in frame 6, physical address = 6 × 1024 + 952 = **7096**.
- **Page table**: one per process, stored in memory and located by the **page-table base register (PTBR)**. Each entry may also have a **valid/invalid bit** (is the page part of the process?), protection bits (read/write/execute) and a dirty bit.
- **TLB (translation look-aside buffer)**: a small, fast associative cache of recent page-table entries. Without it every memory access needs two accesses (page table, then data). Effective access time = h × (TLB + mem) + (1 − h) × (TLB + 2 × mem), where h is the hit ratio. With 20 ns TLB, 100 ns memory and 80% hits: 0.8 × 120 + 0.2 × 220 = **140 ns**.
- **Advantages**: no external fragmentation; easy allocation (any free frame); supports **sharing** (shared code pages) and **protection**; basis for **virtual memory** and demand paging.

- **Disadvantages**: **internal fragmentation** in the last page (on average half a page per process); page tables use memory (solved with multilevel, hashed or inverted page tables); translation overhead without a TLB.
- **Demand paging and page faults**: pages are loaded only when first used; referencing a page not in memory causes a **page fault**, and the OS loads it from disk, using a **page-replacement algorithm** (FIFO, Optimal, LRU) if no frame is free. Too many faults = **thrashing**.

14 Spooling, P2P vs client-server, and locality

Spooling (Simultaneous Peripheral Operations On-Line; Q2a, 4 marks) uses the disk as a large buffer between a fast process and a slow device. Output for a device such as a printer is written to a **spool** file on disk and the device takes jobs from the spool queue one at a time (FCFS). Benefits: programs finish without waiting for the printer; several users can "print" at once; jobs can be queued, reordered or cancelled; the CPU and I/O overlap.

P2P vs client-server (Q1c)	Advantages of peer-to-peer	Disadvantages of peer-to-peer
Points	No central server to buy or fail (no single point of failure); cheap and easy to set up; every peer adds capacity (scales with users, as in file sharing); resources shared directly; suits small offices and ad hoc networks.	Weak central security and access control; no central backup, data scattered; hard to manage and find resources as the network grows; performance depends on peers' machines being on; each user must administer their own machine; harder to guarantee data consistency.

Locality (Q5b, 15 marks). Programs tend to reuse data and instructions they used recently (**temporal locality**) and to access addresses near recent ones (**spatial locality**). Operating systems exploit this in at least these ways:

1. **Caches and the TLB**: recently used memory blocks and page-table entries are kept in fast cache; because the same items are reused (temporal) and neighbours are used next (spatial, whole cache lines are loaded), most accesses hit the cache.
2. **Virtual memory and the working set**: demand paging keeps in RAM only the pages a process is currently using (its **working set**); LRU page replacement evicts the page least recently used, betting that recent use predicts future use. Locality is why paging works without thrashing.
3. **File-system buffer cache and read-ahead**: recently read disk blocks are kept in memory (temporal); when a file is read sequentially the OS **prefetches** the next blocks before they are requested (spatial).
4. Also: disk scheduling and placing related file blocks in the same cylinder group; CPU-affinity scheduling (keeping a process on the same core so its cache stays warm).

15 Exam questions and spotter

PRACTICE QUESTIONS (25 MARKS EACH, ANSWER ANY FOUR, AS IN NOV 2024)

1. (a) Explain the three main purposes of an operating system. [6] (b) Discuss the methods for handling deadlocks. [9] (c) Describe the advantages and disadvantages of peer-to-peer systems over client-server systems. [10]
2. (a) Describe spooling. [4] (b) What are the benefits of multithreaded programming? [5] (c) Explain the concept of paging in detail. [16]
3. (a) Describe FCFS, SJF and Round Robin. [15] (b) For a given table of processes, calculate the average waiting time for FCFS, SJF and Priority scheduling. [10]
4. (a) Difference between a job and a process. [5] (b) Explain multiprogramming and give five advantages. [10] (c) What is process scheduling? [2] Characteristics of a good scheduler and criteria affecting its performance. [8]
5. (a) Explain pseudo-parallelism [2] and the process model [8]. (b) Describe three ways an OS exploits locality. [15]
6. Draw and explain the five-state process diagram and the contents of a PCB. [15]
7. Processes P1 (arr 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5): draw Gantt charts and find the average waiting time for SJF and SRTF. [10]
8. Explain how the size of the time quantum affects Round Robin performance. [6]

Topic	Chance	Why
Scheduling calculations (Gantt + averages)	HOT	10 marks every year; easy marks if methodical
Describe FCFS, SJF, RR (and priority)	HOT	15 marks on Nov 2024
Paging (with diagram and address example)	HOT	16 marks: the largest single part
Deadlock conditions and handling	HOT	Core topic in every OS course
Process states, PCB, threads	LIKELY	Foundation of scheduling questions
Multiprogramming; job vs process; schedulers	LIKELY	Q4 on Nov 2024
Locality; pseudo-parallelism; spooling; P2P	LIKELY	Q1, Q2, Q5 on Nov 2024

EXAMINER'S TIPS FOR SCHEDULING QUESTIONS

- Always draw the Gantt chart with times under **every** boundary, starting at 0; marks are given for the chart.
- Show a table of WT (and TAT) per process, then the sum ÷ n. Write the formula once.
- State your assumptions: arrival 0 if not given; lower number = higher priority; ties broken FCFS; non-preemptive unless told otherwise.
- Units: ms. Check: in non-preemptive schedules with arrival 0, the last completion time equals the sum of all bursts.
- End with one sentence comparing the algorithms.

References

Coffman, E.G., Elphick, M.J. and Shoshani, A. (1971) 'System deadlocks', *ACM Computing Surveys*, 3(2), pp. 67–78.

National University of Science and Technology (2024) *SCS 1103 Operating System*, Final Examination Paper, November 2024. Bulawayo: NUST.

Silberschatz, A., Galvin, P.B. and Gagne, G. (2018) *Operating System Concepts*. 10th edn. Hoboken, NJ: Wiley.

Stallings, W. (2018) *Operating Systems: Internals and Design Principles*. 9th edn. Harlow: Pearson.

Tanenbaum, A.S. and Bos, H. (2015) *Modern Operating Systems*. 4th edn. Harlow: Pearson.